Simulating data from statistical models

MATH2002/3

Richard Mann

School of Mathematics, University of Leeds
r.p.mann@leeds.ac.uk

Today

How do we generate simulated data from a model, using random numbers on a computer?

1. Why simulate at all? A first example: linear regression
2. Generative models
3. Random and pseudo-random numbers
4. Random number generators in R
5. Simulating data from a model: Monte Carlo
6. Sampling from a finite population
7. Assignment 1

Why simulate?

Why simulate data from a model?

- An "intuition pump" for understanding statistics
- Checking that analytical methods perform as expected
- Dealing with complex models that don't have "nice" mathematical properties

Today we will mostly use simulation to build a deeper understanding of **statistical sampling theory**.

(Those doing projects in semester 2 may find other uses...)

How do you know linear regression works?

In MATH1700 you fitted straight lines by least squares. We assume

$$y = a + bx + \epsilon,$$

a real linear relationship between predictor x and response y, corrupted by a residual ϵ.

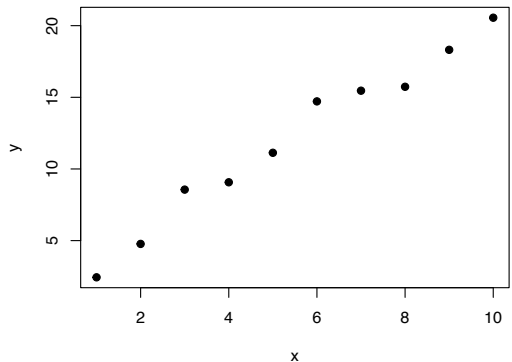
- We estimate a and b by minimising the sum of squared residuals.
- We can *prove* these estimates are unbiased...
- ...but it can be more illuminating to see that the method works.

Idea: make a data set where we *know* the answer ($a = 1$, $b = 2$), then see if `lm()` recovers it.

Step 1: create the data

```
a = 1
b = 2
x = 1:10
epsilon = rnorm(n=10, mean=0, sd=1)
y = a + b*x + epsilon
plot(x, y)
```

- We need $n = 10$ random residuals: one per data point.
- They come from a normal distribution (more on this later).



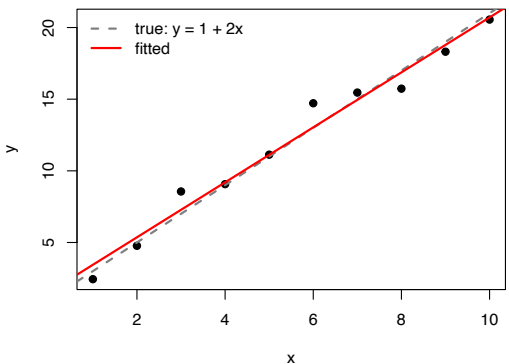
Step 2: fit the model

```
m = lm(y ~ x)
print(summary(m))
```

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   1.5255     0.6673   2.286   0.0516 .
x             1.9180     0.1075  17.835    1e-07 ***
---
Residual standard error: 0.9768 on 8 degrees of freedom
Multiple R-squared:  0.9755,  Adjusted R-squared:  0.9724
```

- True $b = 2$: estimate $\hat{b} = 1.918$ — quite close
- True $a = 1$: estimate $\hat{a} = 1.526$ — a little further off

True line vs fitted line



This is *one* data set. A different draw of ϵ gives different estimates.

Think about

- Are the estimates right on average?
- What happens with noisier data?
- With larger or smaller data sets?
- How would you *investigate* this?

(One of the assignment tasks explores this systematically.)

Generative models

Generative models

You may have met "generative AI": Large Language Models such as ChatGPT or Copilot.

- Generative means designed not primarily to *analyse* data, but to *generate* new outputs.
- We won't study LLMs today, but we will use generative models to produce new outputs: simulated data.

A statistical model as a recipe

The usual description of linear regression:

$$Y = a + bX + \epsilon$$

- Y: dependent / response variable
- X: independent / explanatory variable
- a: intercept, b: gradient
- ϵ: the residual — the gap between what the model predicts and reality

Another reading: this is a specification of how Y is generated. If we know X, take $a + bX$ and then *add a residual*.

What should the residual be?

We must specify a distribution for the residuals. A common choice:

$$\epsilon \sim N(0, \sigma)$$

(normal, mean zero, standard deviation σ).

Instructions for generating Y

1. Multiply the value of X by b
2. Add a
3. Draw a residual ϵ from a normal distribution with mean 0 and s.d. σ
4. Add ϵ to $a + bX$ to get Y

Random and pseudo-random numbers

What do we mean by "random"?

- Is a coin toss random? A roll of a die?
- We often *treat* them as random — but each is determined by the laws of physics.
- If we knew enough about how the coin was tossed, perhaps we could predict the outcome with certainty.

Unpredictable $\neq$ random?

Truly random processes

Some physical processes are (as far as we know) truly random.

- **Radioactive decay:** if you can predict exactly when a nucleus will decay, write it up and prepare to win a Nobel Prize.
- **Premium Bonds:** winners are chosen each month by ERNIE (Electronic Random Number Indicator Equipment), which uses quantum technology to generate *truly* random numbers.

For our purposes this is too complex and too slow.

Pseudo-random numbers

What we actually want are numbers that are

- unpredictable to us, and
- follow a specific distribution.

A pseudo-random number generator (RNG) is a *deterministic* algorithm producing a sequence of numbers in a very complex, unpredictable pattern.

- Many algorithms exist; R (and many other languages) use the Mersenne Twister.
- We won't study the algorithm — see Wikipedia if curious.

Setting the seed

The sequence from an RNG depends on how it is initialised: the seed.

- Do nothing, and most languages set it from the system clock.
- Set it *manually* and re-running your code gives the same results.

```
set.seed(123)
runif(3)
runif(3)
set.seed(123)
runif(3)
```

```
[1] 0.2875775 0.7883051 0.4089769
[1] 0.8830174 0.9404673 0.0455565
[1] 0.2875775 0.7883051 0.4089769
```

Same seed $\Rightarrow$ same "random" numbers. Good for reproducibility.

Random number generators in R

Random number generators

- R, Python, C, Java... all include random number generators.
- The most fundamental generate random integers between 0 and some maximum N.
- These are then transformed to give numbers obeying other distributions.
- R provides functions for a wide variety of distributions, all with a similar syntax:

`rdist(n, parameters)`

how many numbers to generate, then the parameters of the distribution

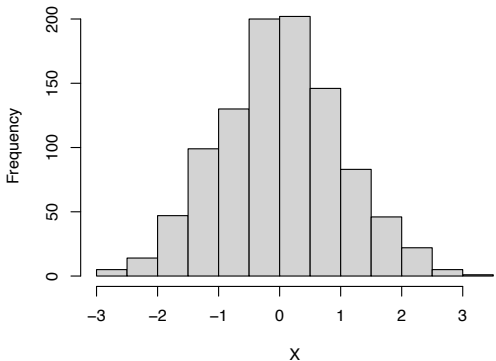
Example: the normal distribution

$n = 1000$ draws from $N(0, 1)$:

```
X = rnorm(n=1000, mean=0, sd=1)
```

Check with a histogram (`breaks` = number of bins):

```
hist(X, breaks=10,
     xlab='X', ylab='Frequency')
```



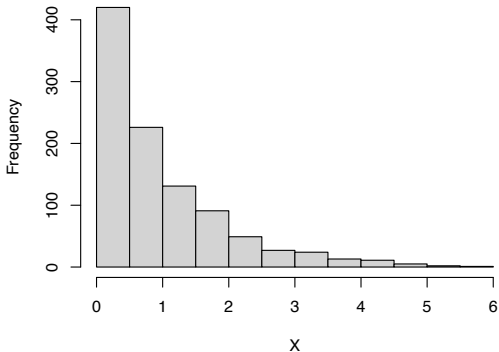
Example: the exponential distribution

$n = 1000$ draws, mean 1:

```
X = rexp(n=1000, rate=1)
hist(x=X, breaks=10,
     xlab='X', ylab='Frequency')
```

The exponential has one parameter, the rate:

$$\text{rate} = \frac{1}{\text{mean}}$$



Common distributions in R

Function	Distribution
`runif(n, a, b)`	continuous uniform between a and b
`rnorm(n, m, sd)`	normal, mean m, s.d. sd
`rexp(n, r)`	exponential, rate (inverse mean) r
`rgeom(n, q)`	geometric, probability q
`rbinom(n, N, q)`	binomial, sample size N, probability q
`rchisq(n, df)`	chi-squared, df degrees of freedom

- Others are easy to find: try searching "gamma distribution R".
- Some need a package, e.g. the multivariate normal is in `mvtnorm`.

Simulating data from a model

Monte Carlo methods

Random numbers let us simulate data from a statistical model, to check:

- does the model produce data that look like real data?
- how well does our analysis perform on data sets of different sizes and types?

This is a Monte Carlo method, named by Stanislaw Ulam after the casino in Monaco.



Monte Carlo Casino



Stanislaw Ulam

Example: heights of men and women

Simulate heights for n men and m women (broadly accurate UK figures):

	Mean (cm)	**s.d. (cm)**
Men	176	7
Women	162	6

- Start with $n = m = 10$.
- True difference in means: $176 - 162 = 14\,\text{cm}$.
- Question: how well does a sample of this size estimate that difference?

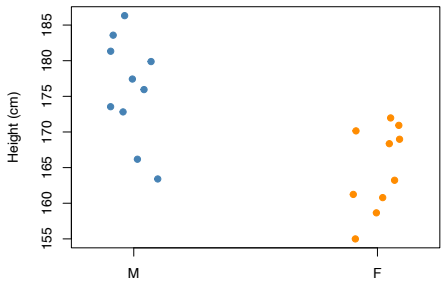
Simulating one data set

Indicator variable for sex:

```
n = 10
m = 10
Sex = c(rep('M', n), rep('F', m))
```

Blank vector of NAs, then fill it in:

```
Height = rep(NA, n+m)
Height[Sex=='M'] = rnorm(n, mean=176, sd=7)
Height[Sex=='F'] = rnorm(m, mean=162, sd=6)
```



Estimating the difference

Now imagine we have *received* this data and are asked to estimate the average height difference.

Best estimate: the difference in the two sample means.

```
D = mean(Height[Sex=='M']) - mean(Height[Sex=='F'])
print(D)
```

```
[1] 11.11343
```

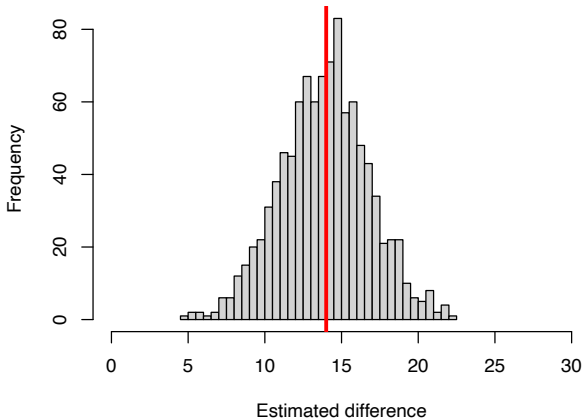
- The true value is 14 cm.
- Our estimate depends on the data set. How precise is it?
- Repeat the whole process many times!

Repeat 1000 times

```
D = rep(NA, 1000)
for (i in 1:1000){
  Height = rep(NA, n+m)
  Height[Sex=='M'] = rnorm(n, mean=176, sd=7)
  Height[Sex=='F'] = rnorm(m, mean=162, sd=6)
  D[i] = mean(Height[Sex=='M']) - mean(Height[Sex=='F'])
}
hist(D, breaks=30, xlab='Estimated difference',
     ylab='Frequency', main="", xlim=c(0, 30))
abline(v=14, col='red', lwd=3)
```

- `abline` draws a thick red line at the true value.
- `xlim` fixes the axis, so later plots are on the same scale.

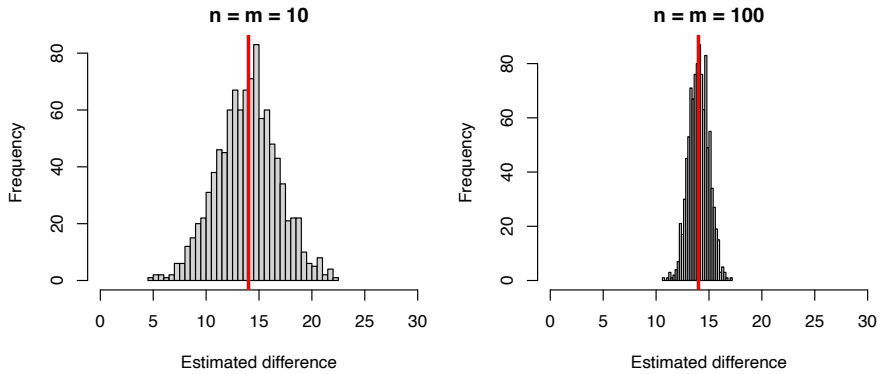
Sampling distribution of D: $n = m = 10$



- Estimates are centred on the true value (red).
- But any *single* estimate could easily be off by several cm.
- What if we collected 10 times more data?

Now with $n = m = 100$

Same code, but with `n = 100`, `m = 100` — ten times more data:



On the same scale, estimates cluster much more closely around the truth.

Sampling from a set

Populations and samples

- In statistics we infer a property of a large population from a smaller sample.
- For convenience, we usually treat the population as effectively infinite.
- Of course, this isn't actually true.

A finite population

What can small samples tell us about, say, all students in the School of Mathematics? (About 1000 students, split roughly equally male and female.)

Step 1: generate the population

500 male and 500 female heights, and the actual population difference:

```
male_heights_population   = rnorm(n=500, m=176, sd=7)
female_heights_population = rnorm(n=500, m=162, sd=6)
D_population = mean(male_heights_population) -
               mean(female_heights_population)
print(D_population)
```

```
[1] 14.21876
```

Note: this is not exactly 14 — it is the true difference *in this finite population*.

Step 2: take a sample

`sample(x, size, replace = FALSE)`

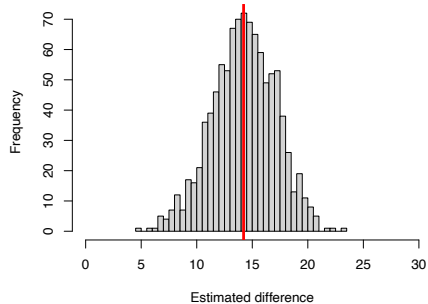
- `x`: the vector to sample from
- `size`: how many to sample
- `replace`: with or without replacement (default: without)

```
male_heights_sample   = sample(x=male_heights_population,
                               size=10, replace=FALSE)
female_heights_sample = sample(x=female_heights_population,
                               size=10, replace=FALSE)
D_sample = mean(male_heights_sample) - mean(female_heights_sample)
print(D_sample)
```

```
[1] 14.97247
```

Step 3: repeat many times

```
D_sample = rep(NA, 1000)
for (i in 1:1000){
  male_heights_sample =
    sample(x=male_heights_population,
           size=10, replace=FALSE)
  female_heights_sample =
    sample(x=female_heights_population,
           size=10, replace=FALSE)
  D_sample[i] = mean(male_heights_sample) -
                mean(female_heights_sample)
}
hist(D_sample, breaks=30, xlim=c(0, 30))
abline(v=D_population, col='red', lwd=3)
```



Red line: the population difference

Why is this useful?

- It mimics the real sampling process from a population more closely than assuming an infinite population.
- It helps make sense of those often-convoluted definitions of confidence intervals and hypothesis tests.

Hypothesis testing, by simulation

Test H_0: the true population mean is zero. Our sample mean is $\bar{X}$.

"If we took a very large number of samples of the same size from a population with true mean zero, how often would the sample mean be $\geq \bar{X}$?"

We can answer this by actually taking lots of samples and looking at their means.

Assignment 1

Assignment 1: what to do

- Complete any two of the four tasks.
- Present your findings in a 2-page report, using the LaTeX template provided.
- Mix text and visualisations — ideally one figure per task — plus a short introduction and conclusion.
- Audience: an incoming first-year undergraduate or final-year A-level student.
- Don't include large amounts of R code; focus on what your investigation shows.
- Relate to your statistics knowledge, but no detailed derivations: it is enough to state known results.

Tasks 1 and 2

Task 1: heights, varying sample size

Repeat the height sampling for $n = m = k$, with $k = 1, 5, 10, 25, 50$. Visualise estimator performance as a function of k. Confirm graphically that the average squared error in `D_sample` scales as $1/k$. What happens when $k = 500$? (Sample *without replacement*!)

Task 2: linear regression

$y = 1 + x + \epsilon$, $\epsilon \sim N(0, 1)$, $x = 0, 1, \ldots, k$ with $k = 10$. Fit with `lm()` and record the slope. Repeat 1000 times and visualise how the estimates cluster around the truth. What changes for different σ in $\epsilon \sim N(0, \sigma)$?

Tasks 3 and 4

Task 3: radioactive decay

1 g of uranium: on average 12,000 decays per second; intervals are exponentially distributed. Simulate $n = 1000$ intervals and record the sample mean. Repeat $k = 1000$ times. Now try $n = 1, 10, 100$. What happens as n gets smaller, and why?

Task 4: estimating π

Draw $n = 10$ points uniformly in the unit square. Count how many satisfy $x^2 + y^2 < 1$. How can this estimate π? Repeat for $n = 50, 100, 500, 1000$ (many times each), visualise how the error changes, and comment on the pattern.

Summary

- A statistical model can be read as a recipe for generating data.
- Computers use pseudo-random number generators; set the seed for reproducibility.
- R has `rdist()` functions for many distributions, and `sample()` for finite sets.
- Monte Carlo: simulate many data sets, apply your estimator to each, and look at the distribution of the results.
- Larger samples $\Rightarrow$ estimates cluster more tightly around the truth.

Questions?

Assignment 1 – *see Minerva*

- Take you own approach, but also discuss ideas with your classmates.
- Use common sense, your own judgement, and your mathematical creativity.
- Help each other, but don't copy.
- You can also use CoPilot as an assistant.
- Start the assignment before the practical next week, so you can ask questions and get help.

The report:

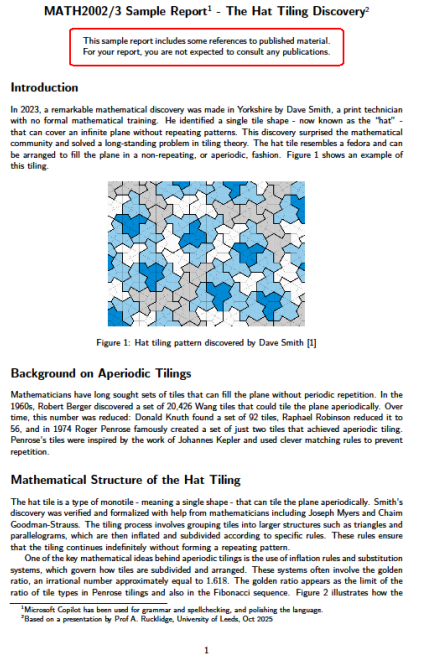
Put yourself into the shoes of your target audience!

- Select carefully what to include, and what not to include.
- The introduction is important. Your target audience does not know anything about the problem.
- Be clear and concise.
- Use professional and precise language
- Structure your report.
- Make a point – give evidence – comment/justify

Attention to Detail

- Free of typos. Free of grammatical errors, using capitalisation & punctuation
- No overly long sentences
- No repetitions
- Consistent formatting
- Clear structure
- No superfluous information

See Minerva for a good and for a poor example of a report on a very different topic, aimed at a different audience (Year 12 & 13 high school students).



MATH2002/3 Sample Report[1] - The Hat Tiling Discovery[2]

This sample report includes some references to published material. For your report, you are not expected to consult any publications.

Introduction

In 2023, a remarkable mathematical discovery was made in Yorkshire by Dave Smith, a print technician with no formal mathematical training. He identified a single tile shape - now known as the "hat" - that can cover an infinite plane without repeating patterns. This discovery surprised the mathematical community and solved a long-standing problem in tiling theory. The hat tile resembles a fedora and can be arranged to fill the plane in a non-repeating, or aperiodic, fashion. Figure 1 shows an example of this tiling.

Figure 1: Hat tiling pattern discovered by Dave Smith [1]

Background on Aperiodic Tilings

Mathematicians have long sought sets of tiles that can fill the plane without periodic repetition. In the 1960s, Robert Berger discovered a set of 20,426 Wang tiles that could tile the plane aperiodically. Over time, this number was reduced: Donald Knuth found a set of 92 tiles, Raphael Robinson reduced it to 56, and in 1974 Roger Penrose famously created a set of just two tiles that achieved aperiodic tiling. Penrose's tiles were inspired by the work of Johannes Kepler and used clever matching rules to prevent repetition.

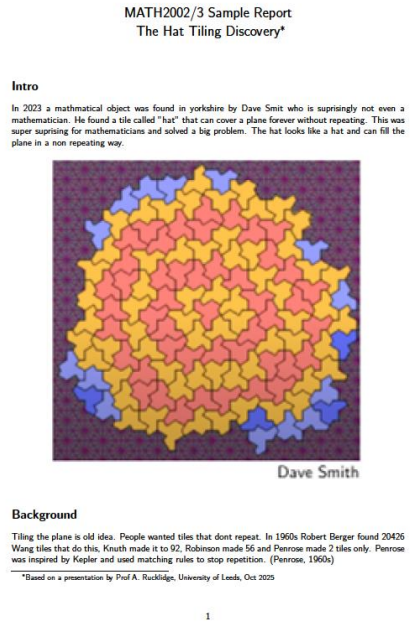
Mathematical Structure of the Hat Tiling

The hat tile is a type of monotile - meaning a single shape - that can tile the plane aperiodically. Smith's discovery was verified and formalized with help from mathematicians including Joseph Myers and Chaim Goodman-Strauss. The tiling process involves grouping tiles into larger structures such as triangles and parallelograms, which are then inflated and subdivided according to specific rules. These rules ensure that the tiling continues indefinitely without forming a repeating pattern.

One of the key mathematical ideas behind aperiodic tilings is the use of inflation rules and substitution systems, which govern how tiles are subdivided and arranged. These systems often involve the golden ratio, an irrational number approximately equal to 1.618. The golden ratio appears as the limit of the ratio of tile types in Penrose tilings and also in the Fibonacci sequence. Figure 2 illustrates how the

[1]Microsoft Copilot has been used for grammar and spellchecking, and polishing the language.
[2]Based on a presentation by Prof A. Rucklidge, University of Leeds, Oct 2025

1



MATH2002/3 Sample Report
The Hat Tiling Discovery*

Intro

In 2023 a mathmatical object was found in yorkshire by Dave Smit who is suprisingly not even a mathematician. He found a tile called "hat" that can cover a plane forever without repeating. This was super suprising for mathematicians and solved a big problem. The hat looks like a hat and can fill the plane in a non repeating way.

Dave Smith

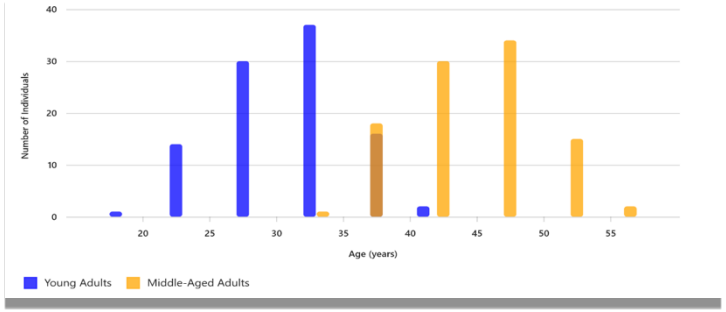
Background

Tiling the plane is old idea. People wanted tiles that dont repeat. In 1960s Robert Berger found 20426 Wang tiles that do this, Knuth made it to 92, Robinson made 56 and Penrose made 2 tiles only. Penrose was inspired by Kepler and used matching rules to stop repetition. (Penrose, 1960s)

*Based on a presentation by Prof A. Rucklidge, University of Leeds, Oct 2025

1

More Attention to Detail



The Maths:

- Treat the maths as part of sentences, including punctuation.
- Format the maths properly, i.e. use math mode in LaTeX , e.g. x rather than x, where appropriate.
- Don’t use maths symbols where words are more appropriate (such as $\rightarrow$ or $\therefore$).

Illustrations/figures/charts:

- Use high resolution images.
- Credit/reference, if downloaded or copied.
- Avoid using screenshots.
- Include a title or caption indicating what is shown.
- Refer to the illustration in the text.
- Pay attention to axis labels, units, legend, colours